

Лекция 3. Циклы – повторение действий

1) Актуальность темы

Циклы избавляют от необходимости писать однотипные команды сотни раз. Они лежат в основе любой обработки последовательностей, опроса датчиков, игровых движков и автоматизации. Умение организовывать повторение – один из ключевых навыков программиста.

2) Цикл while

Цикл while повторяет блок кода, пока условие остаётся истинным:

```
python
count = 0
while count < 5:
    print(count)
    count += 1 # увеличиваем счётчик
```

Если условие никогда не станет ложным, цикл станет бесконечным. Для прерывания используют команду break:

```
python
while True:
    answer = input("Введите 'exit' для выхода: ")
    if answer == "exit":
        break
```

Команда continue прекращает текущую итерацию и сразу переходит к проверке условия, пропуская оставшийся код в теле цикла.

3) Цикл for и функция range()

Цикл for в Python используется для перебора элементов любой последовательности. Чаще всего его применяют вместе с range():

```
python
for i in range(5):
    print(i) # выведет 0, 1, 2, 3, 4
```

range(start, stop, step) генерирует арифметическую прогрессию. stop не включается. Примеры:

```
python
range(1, 6)    # 1, 2, 3, 4, 5
range(10, 0, -1) # 10, 9, ..., 1
```

Цикл for может напрямую перебирать строки:

```
python
for char in "Python":
    print(char)
```

4) Вложенные циклы

Один цикл можно поместить внутрь другого. Это удобно для обхода двумерных структур, например таблицы умножения:

python

```
for i in range(1, 4):
```

```
    for j in range(1, 4):
```

```
        print(f'{i} * {j} = {i*j}')
```

Внутренний цикл полностью выполняется на каждой итерации внешнего. При этом `break` прерывает только ближайший цикл, в котором находится.

5) Типичные приёмы и ошибки

- Всегда изменяйте условие внутри `while`, чтобы избежать зависания.
- Не путайте `=` (присваивание) и `==` (сравнение) в условии цикла.
- В цикле `for` переменная существует после завершения цикла и содержит последнее значение.

Итоги лекции (что нужно запомнить)

- `while` повторяет блок, пока условие истинно; `break` – экстренный выход, `continue` – пропуск итерации.
- `for` перебирает последовательности; `range()` создаёт последовательность чисел.
- Вложенные циклы обрабатывают многомерные данные.